FIAP – FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANA CAROLINA PEREIRA FONTES
JOÃO VICTOR NASCIMENTO ADÃO
JOHNNY DIAS MATHIAS JUNIOR
LUISA GANASEVICI DE ABREU
MATHEUS MOYA DE OLIVEIRA

SUPERBICHO: INTEGRAÇÃO MULTIDISCIPLINAR

plataforma de gestão da saúde de pets

SÃO PAULO
2026

Ana Carolina Pereira Fontes – RM 562145
João Victor Nascimento Adão – RM 563409
Johnny Dias Mathias Junior – RM 566516
Luisa Ganasevici de Abreu – RM 563403
Matheus Moya de Oliveira – RM 562822

SUPERBICHO: INTEGRAÇÃO MULTIDISCIPLINAR

plataforma de gestão da saúde de pets

Documento de integração multidisciplinar apresentado como parte da entrega final (4ª Sprint) do Challenge do curso de Tecnologia em Análise e Desenvolvimento de Sistemas da FIAP, na disciplina de Java Advanced.

SÃO PAULO
2026

SUMÁRIO

1 INTRODUÇÃO 4

2 MAPA DE INTEGRAÇÃO DAS DISCIPLINAS 4

3 MASTERING RELATIONAL AND NON-RELATIONAL DATABASE 5

3.1 Modelo de dados 5

3.2 Restrições e integridade referencial 5

3.3 Versionamento do esquema com Flyway 7

3.4 Integração do banco com a aplicação 8

3.5 Modelo relacional e não relacional 8

4 DEVOPS TOOLS & CLOUD COMPUTING 9

4.1 Contêiner Docker em múltiplos estágios 9

4.2 Publicação em nuvem 9

4.3 Versionamento e fluxo de entrega 9

5 COMPLIANCE, QUALITY ASSURANCE & TESTS 11

5.1 Testes automatizados 11

5.2 Testes de segurança da API 11

5.3 Controles de segurança e privacidade 12

6 JAVA ADVANCED 13

7 CONCLUSÃO 13

1 INTRODUÇÃO

O SuperBicho é uma plataforma web que centraliza a rotina de saúde dos animais de estimação e conecta três perfis de usuário: os tutores, que cadastram seus animais, agendam consultas e exames e acompanham a vacinação; as clínicas veterinárias, que gerenciam a agenda e os veterinários; e o administrador, que aprova as clínicas parceiras.

O cuidado preventivo é o centro da proposta. O sistema calcula automaticamente a situação de cada vacina (atrasada, nunca aplicada ou próxima do vencimento), apresenta alertas sazonais ao tutor e registra o motivo de todo cancelamento, de modo que tutor e clínica saibam sempre o que aconteceu com cada atendimento.

Este documento apresenta como as disciplinas do semestre foram aplicadas na construção da solução e indica onde estão as evidências de cada uma no repositório do projeto (https://github.com/luisa-ganasevici/SuperBicho) e na aplicação publicada (https://superbicho.onrender.com).

2 MAPA DE INTEGRAÇÃO DAS DISCIPLINAS

O Quadro 1 resume a contribuição de cada disciplina e a evidência correspondente. As seções seguintes detalham cada uma delas.

Quadro 1 – Disciplinas aplicadas no SuperBicho

Disciplina	**Aplicação no SuperBicho**	**Evidência**
Mastering Relational and Non-Relational Database	Modelagem relacional em Oracle com 12 tabelas, restrições e integridade referencial, versionada em 11 migrations Flyway.	src/main/resources/db/migration; Figura 1
DevOps Tools & Cloud Computing	Container Docker multi-stage, publicação em nuvem (Render), credenciais em variáveis de ambiente e fluxo de branches com Pull Requests.	Dockerfile; aplicação publicada; histórico do GitHub
Compliance, Quality Assurance & Tests	62 testes automatizados (JUnit 5 e Mockito), testes de segurança da API no Postman e controles de acesso aos dados.	src/test; docs/SuperBicho.postman_collection.json
Java Advanced	Aplicação Spring Boot: MVC com Thymeleaf, Spring Security com três perfis, JPA, Flyway, cache, validações e API REST.	Código-fonte do repositório

3 MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

O banco de dados do SuperBicho utiliza o Oracle Database disponibilizado pela FIAP e foi modelado segundo o paradigma relacional. A escolha se justifica pela natureza dos dados: tutores, animais, clínicas, consultas e vacinas são entidades fortemente relacionadas, e a aplicação depende de integridade referencial entre elas.

3.1 Modelo de dados

O Quadro 2 apresenta as tabelas do modelo e a função de cada uma.

Quadro 2 – Tabelas do banco de dados

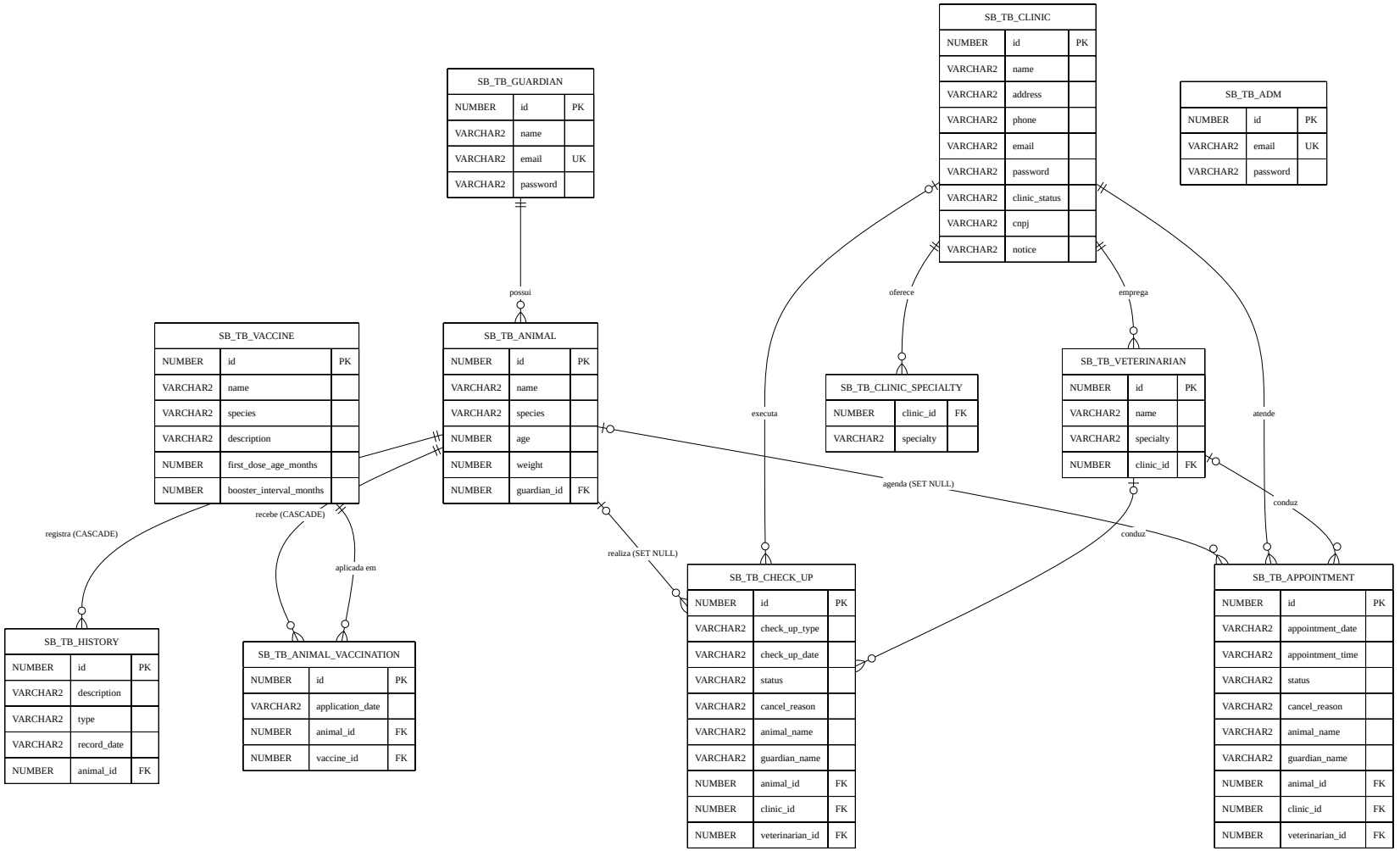
Tabela	**Função**
SB_TB_GUARDIAN	Tutores (e-mail único e senha armazenada em hash BCrypt)
SB_TB_ANIMAL	Animais, vinculados ao tutor
SB_TB_CLINIC e SB_TB_CLINIC_SPECIALTY	Clínicas, situação de aprovação, CNPJ e especialidades
SB_TB_VETERINARIAN	Veterinários de cada clínica
SB_TB_APPOINTMENT	Consultas: data, horário, situação e motivo de cancelamento
SB_TB_CHECK_UP	Exames, com o mesmo controle de situação
SB_TB_HISTORY	Histórico de eventos do animal
SB_TB_VACCINE e SB_TB_ANIMAL_VACCINATION	Catálogo de vacinas por espécie e aplicações registradas
SB_TB_ADM	Administradores (sem cadastro público)

3.2 Restrições e integridade referencial

A Figura 1 apresenta o diagrama entidade-relacionamento resultante de todas as migrations.

- Chaves primárias geradas com GENERATED BY DEFAULT AS IDENTITY em todas as tabelas.
- Restrição de unicidade do e-mail de tutores e administradores (uk_guardian_email e uk_admin_email).
- Chaves estrangeiras ligando o animal ao tutor, a consulta e o exame à clínica e ao veterinário, e a vacinação ao animal e à vacina.
- Regras de exclusão definidas pela regra de negócio: o histórico e as vacinas do animal usam ON DELETE CASCADE; consultas e exames usam ON DELETE SET NULL e guardam o nome do animal e do tutor, para que a clínica continue vendo o registro mesmo após a exclusão do animal.

Figura 1 – Diagrama entidade-relacionamento do SuperBicho (após a migration V11)



3.3 Versionamento do esquema com Flyway

Nenhuma alteração de banco é feita manualmente: cada mudança é um script versionado, aplicado automaticamente pelo Flyway na inicialização da aplicação, inclusive no ambiente publicado. O Hibernate não altera o esquema (ddl-auto: none), de modo que o banco é sempre o resultado exato das migrations, conforme o Quadro 3.

Quadro 3 – Migrations do projeto

Versão	**Tipo de operação**	**Descrição**
V1	DDL – criação	Esquema inicial: tutor, animal, clínica, consulta, exame e histórico
V2	DDL – alteração	Remove o tipo de usuário, adiciona o login da clínica e cria a tabela de administrador
V3	DML – carga	Insere o administrador inicial
V4	DDL – renomeação	Renomeia a coluna de situação da clínica
V5	DDL – alteração	Adiciona o CNPJ e a tabela de especialidades
V6	DDL – criação e FKs	Veterinários, motivo de cancelamento e clínica no exame
V7	DDL – alteração	Especialidade do veterinário
V8	DDL – alteração	Campo de aviso para a clínica
V9	DML – limpeza	Remove um registro inconsistente (clínica recusada sem CNPJ)
V10	DDL e DML	Catálogo de vacinas com carga inicial e registro de aplicações
V11	DDL e DML	Redefine as FKs do animal (CASCADE e SET NULL) e preenche os nomes nas consultas e exames existentes

A migration V11, reproduzida na Figura 2, ilustra a evolução de um esquema já em produção: adiciona colunas, preenche os dados existentes com subconsultas correlacionadas e recria as chaves estrangeiras com novas regras de exclusão.

Figura 2 – Trecho da migration V11

```
ALTER TABLE SB_TB_APPOINTMENT ADD (animal_name VARCHAR2(255),
                                   guardian_name VARCHAR2(255));

UPDATE SB_TB_APPOINTMENT ap SET
    animal_name   = (SELECT a.name FROM SB_TB_ANIMAL a
                     WHERE a.id = ap.animal_id),
    guardian_name = (SELECT g.name FROM SB_TB_ANIMAL a
                     JOIN SB_TB_GUARDIAN g ON g.id = a.guardian_id
                     WHERE a.id = ap.animal_id);

ALTER TABLE SB_TB_APPOINTMENT MODIFY (animal_id NULL);
ALTER TABLE SB_TB_APPOINTMENT DROP CONSTRAINT fk_appointment_animal;
ALTER TABLE SB_TB_APPOINTMENT ADD CONSTRAINT fk_appointment_animal
    FOREIGN KEY (animal_id) REFERENCES SB_TB_ANIMAL(id) ON DELETE SET NULL;

ALTER TABLE SB_TB_HISTORY DROP CONSTRAINT fk_history_animal;
ALTER TABLE SB_TB_HISTORY ADD CONSTRAINT fk_history_animal
    FOREIGN KEY (animal_id) REFERENCES SB_TB_ANIMAL(id) ON DELETE CASCADE;
```

3.4 Integração do banco com a aplicação

- JPA/Hibernate mapeia cada tabela para uma entidade Java (@Entity, @ManyToOne e @ElementCollection).
- Spring Data JPA gera consultas a partir do nome dos métodos, como findByAnimal_GuardianId.
- Uma consulta JPQL de atualização em lote (@Modifying e @Query) cancela de uma só vez as consultas em aberto de um animal.
- O controle transacional (@Transactional) garante que cancelar as consultas e excluir o animal ocorram juntos ou não ocorram.

3.5 Modelo relacional e não relacional

O SuperBicho utiliza exclusivamente o modelo relacional. Os dados têm estrutura fixa e dependem de relacionamentos e de integridade referencial entre as entidades, cenário em que um banco relacional é mais adequado do que um banco orientado a documentos.

4 DEVOPS TOOLS & CLOUD COMPUTING

A aplicação é empacotada em contêiner e publicada em nuvem, de forma que o mesmo artefato é executado no ambiente local e no ambiente de produção.

4.1 Contêiner Docker em múltiplos estágios

O Dockerfile, apresentado na Figura 3, separa a compilação da execução.

Figura 3 – Dockerfile do projeto

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY gradlew .
COPY gradle gradle
COPY build.gradle settings.gradle ./
RUN chmod +x gradlew
RUN ./gradlew dependencies --no-daemon
COPY src src
RUN ./gradlew bootJar --no-daemon -x test

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- Estágio de compilação com JDK 21: gera o arquivo .jar com o Gradle Wrapper. As dependências são baixadas antes da cópia do código-fonte, aproveitando o cache de camadas do Docker.
- Estágio de execução apenas com JRE 21: a imagem final contém somente o necessário para executar o .jar, ficando menor e com menor superfície de ataque.

4.2 Publicação em nuvem

- A aplicação está publicada no Render (https://superbicho.onrender.com), que gera a imagem a partir do Dockerfile do repositório.
- As credenciais do banco não ficam no código: são lidas das variáveis de ambiente DB_USER e DB_PASSWORD, configuradas na plataforma.
- A aplicação utiliza a porta informada pela plataforma (${PORT:8080}).
- O Flyway aplica automaticamente as alterações de banco a cada nova versão publicada.

4.3 Versionamento e fluxo de entrega

- O código é versionado no GitHub com as branches main (produção) e develop (integração), além de branches de correção (fix/...).
- As alterações chegam à branch main por Pull Request, e a publicação é feita a partir dela.
- O Gradle Wrapper garante a mesma versão de compilação em qualquer máquina, sem instalação prévia.

5 COMPLIANCE, QUALITY ASSURANCE & TESTS

5.1 Testes automatizados

O projeto possui 62 testes automatizados escritos com JUnit 5 e Mockito, organizados no padrão Arrange–Act–Assert. Os testes de serviço isolam a regra de negócio dos repositórios por meio de dublês de teste. O Quadro 4 resume o que cada classe de teste valida. A execução é feita com o comando ./gradlew test.

Quadro 4 – Classes de teste e respectivas validações

Classe de teste	**O que valida**
AnimalServiceTest	Cadastro de animal; busca e edição permitidas apenas ao próprio tutor (403 para outro tutor)
AnimalRemovalServiceTest	Exclusão do animal cancela consultas e exames em aberto, na ordem correta; bloqueio para outro tutor
AppointmentServiceTest	Agendamento, alteração e cancelamento apenas pelo dono; situação inicial sempre AGENDADO; nomes mantidos após a exclusão do animal
CheckUpServiceTest	Exames: cancelamento pelo tutor e pela clínica, conclusão e cancelamento em lote
VaccinationServiceTest	Calendário de vacinas: vencida, próxima do vencimento, em dia e reforço mais recente
ClinicServiceTest, GuardianServiceTest, VeterinarianServiceTest, HistoryServiceTest e AdmServiceTest	Cadastros, aprovação de clínicas e regras de cada perfil
SuperBichoApplicationTests	Inicialização completa da aplicação, incluindo a conexão com o banco e as migrations

5.2 Testes de segurança da API

Uma coleção do Postman executa em sequência o fluxo completo da API, com asserções automáticas em cada requisição. A coleção captura o token CSRF, a sessão e os identificadores criados, e simula um tutor tentando acessar os dados de outro, conforme o Quadro 5.

Quadro 5 – Cenários de teste de segurança da API

Cenário	**Resultado esperado**
Tutor lista, cria, edita e exclui os próprios animais e consultas	200, 201 ou 204
Alteração de consulta já cancelada	400
Tutor B tenta ver, editar ou excluir o animal do tutor A	403
Tutor B tenta ver, remarcar ou cancelar a consulta do tutor A	403
Tutor B agenda consulta para o animal do tutor A	403
Envio de outro dono (guardianId) ou de outra situação no corpo da requisição	Ignorado: dono e situação definidos pelo servidor
Acesso à API sem login, ou com login de clínica ou de administrador	Redirecionamento para o login do tutor

5.3 Controles de segurança e privacidade

- Senhas armazenadas com hash BCrypt, nunca em texto puro.
- Três perfis isolados (tutor, clínica e administrador), cada um com sua cadeia de segurança e sessão própria.
- Acesso restrito aos próprios dados: toda operação sobre animal, consulta ou exame verifica se o recurso pertence ao tutor autenticado.
- Ausência de escalonamento de privilégio: clínicas são criadas com situação pendente e somente o administrador as aprova; não existe cadastro público de administrador.
- Proteção CSRF em todas as requisições que alteram dados.
- Validação de entrada com Bean Validation (campos obrigatórios, e-mail, CNPJ com 14 dígitos, idade e peso válidos).
- Rastreabilidade: consultas e exames cancelados não são apagados; o motivo fica registrado para o tutor e para a clínica.

6 JAVA ADVANCED

A disciplina de Java Advanced é o ponto em que as demais se integram: a aplicação Spring Boot utiliza o banco modelado em Oracle, é empacotada e publicada pelo processo de DevOps e é validada pelos testes e controles de qualidade. O Quadro 6 relaciona os conceitos aplicados.

Quadro 6 – Conceitos de Java Advanced aplicados

Conceito	**Uso no SuperBicho**
Spring MVC e Thymeleaf	Telas dos três perfis renderizadas no servidor
Spring Security	Três SecurityFilterChain, sessão separada por perfil, BCrypt e CSRF
Spring Data JPA	Repositórios, consultas derivadas e JPQL
Flyway	Onze migrations versionadas e aplicadas automaticamente
API REST	Recursos /animals e /appointments com controle de acesso por dono
Cache	Cache das listagens com invalidação nas alterações
Bean Validation	Validação de formulários e de requisições da API
Testes	JUnit 5 e Mockito

7 CONCLUSÃO

O SuperBicho integra as disciplinas do semestre em um único produto funcional. O banco relacional sustenta as regras de negócio com integridade referencial e evolução versionada; as práticas de DevOps publicam a aplicação de forma reproduzível; e os testes e controles de qualidade comprovam que cada perfil acessa somente o que lhe pertence. Tudo isso é entregue pela aplicação construída em Java Advanced.